

Git Training — Speaker's Handbook

Combined Edition: Concept Brush-Up + Slide-by-Slide Script

Companion to Git-Training-By-Sarvesh-Gupta.pptx · 60 minutes · mixed audience (devs + non-devs) · virtual

How this edition is organized

This is Part 2 (the slide-by-slide speaking guide) with the relevant Part 1 concept explanation dropped in directly beneath the slide that calls for it — so you don't have to flip between sections mid-sentence while presenting. Each inserted block is marked  `Concept Reference` and boxed in blue.

Concept sections are merged in at the point they're explicitly invoked in the script: Part 1.4 (Distributed) after Slide 4, Part 1.7 (The four areas) after Slide 6, Part 1.3 (HEAD, branches) after Slide 11, and Part 1.5 (origin & tracking branches) after Slide 15. The remaining Part 1 sections (1.1, 1.2, 1.6) aren't explicitly called out by name anywhere in the Part 2 script, so they aren't duplicated here — see the original handbook's Part 1 if you want to re-read them in full before the session.

PART 2 — Slide-by-slide speaking guide

Each slide gets: the one idea, roughly what to say, and what usually gets asked. Aim for the times on your agenda slide; they're realistic.

Slides 1-2 · Welcome & agenda2 min

Don't over-invest here. Introduce yourself in one line. Then set two expectations that matter for a mixed room:

"Some of you write code every day, some of you have never opened a terminal. I've built this so nobody's lost — the first half is concepts that apply to anyone who works with files, and I'll flag clearly when something is developer-only. Questions in chat any time; I'll take them at the end, or immediately if it's a quick one."

Also say this, because it relaxes everyone: *"The goal today isn't to make you fluent. It's to make sure you understand what your team is talking about, and to give you five commands you can actually use tomorrow."*

Slide 3 · The problem3 min

This is your best non-dev slide. Everyone has lived `report-final-FINAL-v2-USE-THIS.docx`.

Ask the room a question here — it's the right moment for engagement, early:

"Quick one in chat — who's got a file on their machine right now with the word 'final' in the name twice?"

Then land the three failures of the zip-folder approach: you don't know which is current, you can't merge two people's edits, and there's no record of who changed what or why. Git fixes exactly those three.

Slide 4 · What is Git3 min

Keep the history short — Linus Torvalds, 2005, built it in a couple of weeks because the Linux kernel team lost access to the tool they'd been using. It's a nice story but don't spend more than 30 seconds.

Spend your time on the two words that matter: **distributed** (Part 1.4 above) and **version control** (it records history; it doesn't compile, test, or deploy anything).

Worth saying plainly: *"Git doesn't know or care what's in your files. It works on text files, images, spreadsheets, anything. It's just much better at text."*

📖 Concept Reference — Part 1.4called from Slide 4

"Distributed" — what it actually buys you

Older systems (SVN, TFS) kept the history on a central server. You had the latest files; the server had the history. No server, no history, no commits, no diffs.

Git gives every clone the entire history. When you `git clone`, you download every commit ever made. Consequences:

- `git log`, `git diff`, `git commit`, `git branch` all work on a plane with no wifi.
- They're also instant, because nothing crosses a network.
- If GitHub vanished tonight, any developer's laptop could restore the project completely.

The cost: the first clone of a huge, old repository can be slow, and you have to consciously synchronise (push/pull), which is where merge conflicts come from.

"GitHub isn't the source of truth because it's special. It's the source of truth because we all agreed it is. Technically, every laptop in this call has an equally valid full copy."

Slide 5 · Git vs GitHub vs GitLab3 min

The highest-value slide for non-devs. They hear "GitHub" in every standup and think it's the same thing as Git.

"Git is the engine. GitHub is a car built around that engine, with seats and a dashboard. GitLab is a different car with the same engine. There's also Bitbucket, and Azure DevOps. All of them run Git underneath — that's why the commands are identical no matter where your company hosts code."

The practical version: Git works with no internet and no account. GitHub is where you put it so other people can see it, review it, and discuss it.

Slide 6 · The mental model4 min — go slow

Use Part 1.7. Do not rush this slide even if you're behind. Say each transition out loud and point at it:

1. "I edit a file. Git notices, but has done nothing."
2. " `git add` — I'm saying this change belongs in my next save point."
3. " `git commit` — I'm sealing that save point into my local history. Still nobody else can see it."
4. " `git push` — now the team has it."

Then the reverse: " `git pull` brings their work down to me."

The sentence to repeat twice: "**Commit is local. Push is what makes it real for everyone else.**" That confusion causes more "but I committed it!" incidents than anything else on the slide.

📖 Concept Reference — Part 1.7

called from Slide 6

The four areas, said slowly (slide 6)

This is the single most important slide in your deck. Get this right and half the questions never get asked.

Area	What it is	How things get in
Working directory	The actual files on your disk, as your editor sees them	You edit and save
Staging area (index)	A shopping basket for the next commit	<code>git add</code>
Local repository	The <code>.git</code> folder — every commit, on your machine	<code>git commit</code>
Remote repository	GitHub/GitLab — the shared copy	<code>git push</code>

Why the staging area exists at all — this is the question devs ask and most trainers dodge. Answer: because you often change more than one thing before you're ready to commit. Staging lets you commit only the login fix, leave the half-done refactor uncommitted, and keep the two changes as separate commits. It's the difference between "commit everything I've touched" and "commit this one idea."

Non-dev analogy for the whole slide: working directory = your desk, covered in papers. Staging = the envelope you're filling. Commit = sealing and dating the envelope and filing it in your cabinet. Push = couriering a copy to head office.

Slide 7 · The everyday loop

4 min

Frame it as: "If you remember nothing else, remember this cycle. I have used Git professionally for years and this is 90% of what I type."

Emphasise `git status` disproportionately. It is the safest command in Git — it changes nothing, and it tells you exactly where you are and usually what to do next. Tell the room: "*When you're confused, run git status. It's free and it's often the answer.*"

Slide 8 · Setup and first repo

6 min

You have generous time here — use some of it for the first live demo (Part 3). If you'd rather keep demos together, just walk the commands and note:

- The `user.name` and `user.email` you set here get baked into every commit you ever make. Commits are signed with that identity permanently — worth getting right.
- `--global` means "for every repository on this machine." Drop it inside a repo to override just there (useful if you use a work email at work and a personal one for side projects).
- `git init` vs `git clone`: `init` creates a brand-new empty repository in a folder you already have. `Clone` copies an existing one, with all of its history, from a server. You will use `clone` about fifty times for every one time you use `init`.

Slide 9 · The twelve commands

4 min

Don't read all twelve. Read the first five slowly (`status`, `add`, `commit`, `push`, `pull`) and then say: "The rest are on the slide, and I'll come back to branching and merging properly in a moment. Screenshot this."

One useful aside: `git switch` and `git checkout` do overlapping things. `checkout` is the old command that did about six unrelated jobs, which is why it confused everyone. Git split it into `switch` (change branches) and `restore` (undo file changes). Both still work — if the room sees `checkout` in older tutorials, it's not wrong, just older.

Slide 10 · Commit messages

3 min

Good engagement moment — invite worst-commit-message confessions in chat.

Give the practical rule rather than a philosophy: "*Write the message so that it completes the sentence 'If applied, this commit will...'*" That's why imperative present tense — "Add login validation," not "Added login validation." It reads correctly in the log, in release notes, and in the PR list.

And explain atomic in business terms, because non-devs will care: "If a commit does one thing, we can undo exactly that one thing when it breaks in production at 2am. If a commit does five things, we have to undo all five."

Slide 11 · Branching

4 min

Use the bookmark analogy from Part 1.3, then the practical framing:

"A branch is a safe place to be wrong. You can break anything you like on a branch — main is untouched, and nobody sees your mess until you choose to show them."

Mention the naming convention because it's a team decision they'll actually be asked about: `feature/`, `bugfix/`, `hotfix/`, often with a ticket number — `feature/JIRA-1234-login-validation`. The reason is searchability and automation, not aesthetics; CI systems and tools key off those prefixes.

📖 Concept Reference — Part 1.3

called from Slide 11

HEAD, branches, and why branches are "cheap"

This is the concept that unlocks everything on slides 11 and 12.

A branch is not a copy of your code. It is a 41-byte text file containing one SHA. That's literally all it is — a sticky note that says "this branch currently points at commit a1b2c3d."

HEAD is a pointer to which branch you're currently on. When you run `git switch main`, you're moving HEAD to point at main.

When you commit, two things happen: a new commit is created with the current commit as its parent, and the branch sticky-note is moved forward to the new commit.

So "creating a branch" means "writing a 41-byte file." That's why the slide says branches are free. Creating a branch in Git is genuinely faster than creating a folder.

The analogy that lands with non-devs: a branch is a bookmark in a book. Making a new bookmark doesn't copy the book. `git switch` is just moving to a different bookmark.

If someone asks "what's a detached HEAD?" — it means HEAD is pointing directly at a commit rather than at a branch. You get it when you `git checkout <sha>`. Commits you make there belong to no branch, so they're easy to lose. The fix is `git switch -c new-branch-name` to attach a branch to where you are, or `git switch main` to go back.

Slide 12 · Merging

3 min

The one distinction to make clear:

- **Fast-forward:** main hasn't moved since you branched. Git just slides the main bookmark forward to your commit. No merge commit, no conflict, nothing to resolve.
- **Three-way merge:** both branches moved. Git looks at three points — your branch tip, their branch tip, and the common ancestor where you split — and works out a combined result. It succeeds automatically most of the time.

Say clearly: "Git merges by line, not by meaning." Two people editing different lines of the same file merges cleanly with no help. Two people editing the same line is a conflict. That's the whole rule, and it sets up the next slide.

Slide 13 · Merge conflicts

4 min

The most feared topic and the easiest to defuse. Lead with reassurance:

"A conflict is not an error. Nothing is broken and nothing is lost. Git is saying: two people changed the same line, and I'm not willing to guess which one you meant. You decide."

Walk the markers literally:

- `<<<<<<< HEAD` — everything below this is your version (the branch you're on)
- `=====` — the divider
- `>>>>>> feature/dashboard` — everything above this is their version (the branch coming in)

Then the fix: open the file, decide what the final text should be (it might be yours, theirs, or a combination you type fresh), delete all three marker lines, save, test, git add the file, git commit.

Two things worth adding:

- `git merge --abort` puts everything back exactly as it was before you started the merge. It's a free escape hatch, and most people don't know it exists.
- Modern editors (VS Code especially) render conflicts with "Accept Current / Accept Incoming / Accept Both" buttons, so you often never see raw markers. Mention this — it visibly relaxes the room.

Slide 14 · Pull requests

3 min

Reframe it for the non-devs, who will interact with PRs even if they never write code:

"A pull request is a formal request: 'I've done some work on my branch — please review it and, if you're happy, put it into the main'"

project.' It's a discussion thread attached to a set of changes. Reviewers comment, the automated tests run, and when everyone's satisfied, someone clicks merge."

Point out the naming: GitHub says Pull Request, GitLab says Merge Request. Identical thing. GitLab's name is arguably more accurate.

Mention the three merge buttons GitHub offers, since it's a real team decision:

- **Merge commit** — keeps every commit plus a merge commit. Full history.
- **Squash and merge** — flattens your branch's commits into one tidy commit on main. Most popular choice for teams.
- **Rebase and merge** — replays commits onto main with no merge commit.

Slide 15 · Fetch vs pull

3 min

Use the tracking-branch explanation from Part 1.5, then the email analogy on the slide. Repeat the analogy twice — it's on your own speaker notes and it's good advice.

The practical takeaway: "Use pull by default. Use fetch when you want to look before you leap — for example when you know a big change landed upstream and you want to read it before it touches your files."

📖 Concept Reference — Part 1.5

called from Slide 15

The remote, origin, and tracking branches

When you clone a repo, Git saves the URL you cloned from under the nickname `origin`. There's nothing magic about the word — it's just the default nickname for "the place I got this from." You can have several remotes with different names.

Git also keeps a set of read-only remote-tracking branches named `origin/main`, `origin/develop`, and so on. These are Git's memory of where those branches were on the server the last time you talked to it.

That's the key to slide 15:

- `git fetch` updates `origin/main` — Git's memory of the server. Your own `main` is untouched. Nothing in your working files changes.
- `git merge origin/main` then brings that into your branch.
- `git pull` is just those two commands back-to-back.

So when someone says "I pulled and it broke my files" — `pull` is the one that touches your working tree. `Fetch` never does. That distinction alone is worth a minute of the session.

`git push -u origin feature/x` — the `-u` sets up the link between your local `feature/x` and the remote one, so that afterwards plain `git push` and `git pull` know where to go. That's why you only need `-u` the first time you push a new branch.

Slide 16 · Undoing mistakes

4 min

Slow down here; this is the slide that prevents incidents.

The organising question: "Has anyone else seen this commit?"

- **No, it's only on my machine** → `reset` is available to you. `git reset --soft`, `--mixed`, `--hard` move your branch pointer backwards; `--hard` also throws away your file changes.
- **Yes, it's been pushed** → use `git revert`. `Revert` doesn't delete anything. It creates a new commit that is the exact opposite of the old one. History stays intact, everyone's copies stay compatible, and the bad change is neutralised.

Then the safety net nobody knows about, which is a great thing to give a room:

"*git reflog*. Git keeps a private log of everywhere HEAD has been for the last 90 days, including commits you 'deleted'. If you `reset --hard` and panic, `git reflog` usually gets it back. It has saved me and it will save you."

Slide 17 · Log and diff

2 min

Quick slide. The one habit to sell: `git diff --staged` before every commit. It's a five-second self-review that catches debug statements, commented-out code, and accidentally staged files. Frame it as "reading your own work before you submit it."

Useful extras if asked: `git log --oneline --graph --all` draws the branch structure as ASCII art; `git log -- <filename>` shows the history of one file; `git blame <file>` shows who last changed each line (and why — it gives you the commit).

Slide 18 · Case study

5 min

This is where it all clicks. Narrate it as a story, in first person, with a real-sounding ticket:

"It's Monday. I pick up JIRA-1234, add login validation. First thing — `git switch main`, then `git pull`, because I want to start from what the team has, not from Thursday's stale copy. Then I branch: `git switch -c feature/JIRA-1234-login-validation`. I write the code, I test it. `git status` to see what I touched, `git diff` to read it. `git add`, `git commit` with a message that says what it does. `git push -u origin feature/...` I open a PR, tag two reviewers, link the ticket. They comment, I push two more commits addressing their comments, CI goes green, they approve, we `squash-merge`, I delete the branch. Tomorrow morning I `git pull` on main and my own change comes back down to

me as part of the shared history."

That paragraph is the single most useful 90 seconds of the session for developers.

Slides 19-20 · Best practices and anti-patterns

7 min combined

Don't read 20 bullets aloud — you'll lose the room. Pick five you have an opinion about and tell a short story for each. Suggested five:

1. **Never commit secrets.** The critical detail: deleting the file in a later commit does not remove it from history. Anyone who clones the repo gets the whole history, including the commit with your key in it. The only real fix is to rotate the key — assume it's compromised the moment it's pushed. This is the one to say twice.
2. **Pull before you start.** Ninety percent of painful conflicts come from branching off a week-old copy of main.
3. **Small commits.** Ask the room to imagine reviewing a 600-line change vs six 100-line changes. Reviewers skim large PRs; that's where bugs get through.
4. **Don't force-push shared branches.** `--force` rewrites history on the server. Anyone who already pulled now has a history that doesn't match, and their next push either fails or creates a mess. If you truly need it, `--force-with-lease` is the safer version — it refuses if someone else has pushed since you last looked.
5. **Protect main.** On GitHub this is a setting: require a PR, require an approval, require CI to pass. It turns "please don't commit to main" from a team norm into something the tool enforces. Worth noting this is your decision as tech lead, and you can enable it this week.

Also introduce `.gitignore` here if you haven't — it's the file that tells Git "never track these." Typically `node_modules/`, `.env`, `dist/`, `build/`, `.DS_Store`, IDE folders. Non-devs get this instantly: "a list of things not to file."

Slides 21-22 · Demo and cheat sheet

Demo content is Part 3 and Part 4 below. Then tell people explicitly to screenshot slide 22.

Slides 23-24 · Takeaways and Q&A

10-15 min

Read the eight takeaways aloud — it's genuinely the right call for retention. Then open the floor with a specific prompt, because "any questions?" in a virtual room gets silence:

"Let me start with the one I get most often, and then I'll take yours..." — and answer a question from Part 6 yourself. That breaks the ice reliably.